

Workshop 03 – LSTM zur Zeitreihen-Vorhersage auf Forex-Daten

Ziel

Erstelle ein Python-Skript, das Marktdaten aus einer CSV-Datei liest, daraus Sequenzen für ein LSTM-Netz aufbereitet und das Netz so trainiert, dass es aus **64 Zeitschritten** Eingabe die nächsten **32 Zeitschritte** vorhersagt. Das Skript soll trainieren, Vorhersagen erzeugen und beides mit Matplotlib visualisieren.

Datensatz

Verwende `data/AUDCAD_15.csv` (AUD/CAD, 15-Minuten-Kerzen 2010 ff., ca. 350.000 Zeilen, 23 MB). Die Datei enthält die Spalten `v`, `vw`, `o`, `c`, `h`, `l`, `t`, `n` (Volumen, volumengewichteter Preis, Open, Close, High, Low, Zeitstempel, Anzahl Transaktionen). Die Spalte `t` ist ein Datum und wird beim Normalisieren nicht berücksichtigt.

Aufgaben

Arbeite die Funktionen in `workshop.py` ab. Die Stellen sind mit `TODO` markiert.

- Daten einlesen und normalisieren**
 - Lies die Marktdaten aus der CSV-Datei ein (Pandas).
 - Normalisiere jede Spalte individuell (z. B. `MinMaxScaler`).
- Daten aufteilen**
 - Teile die normalisierten Daten in Trainings- und Testdaten auf.
- Datenstruktur für das LSTM erstellen**
 - Erzeuge Arrays mit den Dimensionen (`samples`, `timesteps`, `features`).
 - Verwende 64 Zeitschritte als Eingabe und 32 Zeitschritte als Ausgabe.
- Neuronales Netz erstellen**
 - Entwirf eine LSTM-Architektur, die für die Datenmenge geeignet ist.
 - Die Ausgabeschicht soll 32 Zeitschritte ($\times$ alle Features) gleichzeitig vorhersagen.
- Modelltraining**
 - Trainiere das Modell mit `EarlyStopping` und `ModelCheckpoint`.
 - Plotte die Trainingshistorie (`loss` / `val_loss`) mit Matplotlib.
- Vorhersagen und Visualisierung**
 - Ergänze einen Kommandozeilen-Parameter `--predict`, mit dem das gespeicherte Modell geladen, Vorhersagen auf den Testdaten gemacht und diese gegen die echten Werte geplottet werden.

Bonus (freiwillig)

- Logge das Training mit dem `TensorBoard`-Callback.
- Sage nur die `close`-Spalte statt aller Features vorher und vergleiche die Qualität.

Hinweise

- Bibliotheken: Pandas (Einlesen), Scikit-learn (Normalisieren, Aufteilen), TensorFlow/Keras (Netz), Matplotlib (Plots).
- Achte darauf, dass dein Code gut dokumentiert und kommentiert ist, damit er leicht nachvollziehbar ist.
- Die Ausgabe des Netzes ist ein flacher Vektor der Länge `32 * features`; forme die Labels entsprechend um (`reshape`).